

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор
должность, уч. степень, звание

подпись, дата

А.В. Бржезовский
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №7

ТРИГГЕРЫ

по курсу: Методы и средства проектирования
информационных систем и технологий

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

И.А. Лаврешин
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Изучить механизм триггеров: моменты BEFORE / AFTER, события INSERT / UPDATE / DELETE, обработка FOR EACH ROW, использование OLD и NEW.

2 Задание и исходные данные

Триггеры каждого вида (after, instead of) для каждой из операций (insert, update, delete), обеспечивающие активную целостность данных.

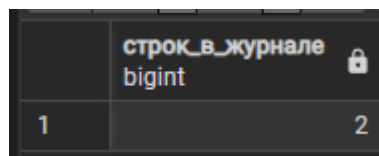
Триггер, реализующий вычисления или формирование статистики или ведение истории изменений в БД.

3 Ход выполнения задания

Создана постоянная таблица lr7_employee_audit с полями для фиксации типа операции, идентификатора сотрудника и старых/новых значений ФИО и даты рождения. Таблица служит приёмником для AFTER-триггера на “сотрудник”. Результаты представлены на рисунках 1 – 2.

```
CREATE TABLE IF NOT EXISTS lr7_employee_audit (  
  id bigserial PRIMARY KEY,  
  ts timestampz NOT NULL DEFAULT clock_timestamp(),  
  op text NOT NULL,  
  emp_id int,  
  fio_old varchar(200),  
  fio_new varchar(200),  
  birth_old date,  
  birth_new date,  
  CONSTRAINT lr7_employee_audit_op_chk CHECK (op IN ('INSERT', 'UPDATE', 'DELETE'))  
);  
  
COMMENT ON TABLE lr7_employee_audit IS 'ЛР7: журнал DML по таблице сотрудник (триггер AFTER).';
```

Рисунок 1 – Процедура “lr7_employee_audit”



строка_в_журнале	bigint
1	2

Рисунок 2 – Результаты работы программы

Создана функция lr7_fn_priyom_date_check с типом возврата trigger и триггер lr7_trg_priyom_date_check с режимом BEFORE INSERT OR UPDATE OF даты приёма на таблице “прием”. Если NEW.дата_приема больше текущей даты, выбрасывается исключение с кодом check_violation. Таким образом, бизнес-правило проверяется до записи строки. Результаты представлены на рисунках 3 – 4.

```

CREATE OR REPLACE FUNCTION lr7_fn_priyom_date_check()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.дата_приема > CURRENT_DATE THEN
        RAISE EXCEPTION 'lr7: дата приёма не может быть позже текущей даты (получено: %)', NEW.дата_приема
        USING ERRCODE = '23514'; -- check_violation
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS lr7_trg_priyom_date_check ON прием;

CREATE TRIGGER lr7_trg_priyom_date_check
BEFORE INSERT OR UPDATE OF дата_приема ON прием
FOR EACH ROW
EXECUTE PROCEDURE lr7_fn_priyom_date_check();

```

Рисунок 3 – Процедура “lr7_fn_priyom_date_check”

	демонстрация text	статус text	сообщение text
1	дата приёма в будущем	отклонено (ожидаемо)	lr7: дата приёма не может быть позже текущей даты (получено: 2026-05-20)

Рисунок 4 – Результаты работы программы

Функция lr7_fn_employee_audit срабатывает AFTER INSERT, UPDATE и DELETE. Для DELETE в журнал пишутся старые значения из OLD, для UPDATE — пара OLD/NEW, для INSERT — значения из NEW. Триггер демонстрирует типичный аудит изменений без изменения основной логики приложения. Результаты представлены на рисунках 5 – 6.

```

CREATE OR REPLACE FUNCTION lr7_fn_employee_audit()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        INSERT INTO lr7_employee_audit (op, emp_id, fio_old, birth_old)
        VALUES ('DELETE', OLD.ид, OLD.фio, OLD.дата_рождения);
        RETURN OLD;
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO lr7_employee_audit (op, emp_id, fio_old, fio_new, birth_old, birth_new)
        VALUES ('UPDATE', NEW.ид, OLD.фio, NEW.фio, OLD.дата_рождения, NEW.дата_рождения);
        RETURN NEW;
    ELSIF TG_OP = 'INSERT' THEN
        INSERT INTO lr7_employee_audit (op, emp_id, fio_new, birth_new)
        VALUES ('INSERT', NEW.ид, NEW.фio, NEW.дата_рождения);
        RETURN NEW;
    END IF;
    RETURN NULL;
END;
$$;

DROP TRIGGER IF EXISTS lr7_trg_employee_audit ON сотрудник;

CREATE TRIGGER lr7_trg_employee_audit
AFTER INSERT OR UPDATE OR DELETE ON сотрудник
FOR EACH ROW
EXECUTE PROCEDURE lr7_fn_employee_audit();

```

Рисунок 5 – Процедура “lr7_fn_employee_audit”

	строк_в_журнале	bigint	
1			4

Рисунок 6 – Результаты работы программы

Перед вставкой в таблицу “дети” триггер lr7_trg_deti_limit подсчитывает уже существующие строки для NEW.ид_сотрудника. Если их не меньше 12, вставка запрещается с сообщением об ошибке. Используется только запись NEW. Результаты представлены на рисунках 7 – 8.

```
CREATE OR REPLACE FUNCTION lr7_fn_deti_limit()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_cnt int;
    v_max constant int := 12;
BEGIN
    SELECT COUNT(*)::int INTO v_cnt
    FROM дети
    WHERE ид_сотрудника = NEW.ид_сотрудника;

    IF v_cnt >= v_max THEN
        RAISE EXCEPTION 'lr7: у сотрудника % уже % детей в таблице (лимит %)',
            NEW.ид_сотрудника, v_cnt, v_max
        USING ERRCODE = '23514';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS lr7_trg_deti_limit ON дети;

CREATE TRIGGER lr7_trg_deti_limit
BEFORE INSERT ON дети
FOR EACH ROW
EXECUTE PROCEDURE lr7_fn_deti_limit();
```

Рисунок 7 – Процедура “lr7_fn_deti_limit”

	ид	фio	детей_в_таблице
	[PK] integer	character varying (200)	integer
1	6	Большая Семья Инженер	7

Рисунок 8 – Результаты работы программы

Перед удалением строки из “подразделение” функция lr7_fn_subdept_delete_guard проверяет наличие дочернего подразделения с ид_родительское = OLD.ид. При наличии дочерней записи удаление

блокируется с поясняющим текстом (удобнее для пользователя, чем только сообщение СУБД о нарушении FK). Результаты представлены на рисунках 9 и 10.

```
CREATE OR REPLACE FUNCTION lr7_fn_subdept_delete_guard()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_child int;
BEGIN
    SELECT п.ид INTO v_child
    FROM подразделение п
    WHERE п.ид_родительское = OLD.ид
    LIMIT 1;

    IF v_child IS NOT NULL THEN
        RAISE EXCEPTION 'lr7: подразделение % (%) нельзя удалить: есть дочернее подразделение с ид %',
            OLD.ид, OLD.наименование, v_child
        USING ERRCODE = '23503'; -- foreign_key_violation
    END IF;

    RETURN OLD;
END;
$$;

DROP TRIGGER IF EXISTS lr7_trg_subdept_delete_guard ON подразделение;

CREATE TRIGGER lr7_trg_subdept_delete_guard
BEFORE DELETE ON подразделение
FOR EACH ROW
EXECUTE PROCEDURE lr7_fn_subdept_delete_guard();
```

Рисунок 9 – Процедура “lr7_fn_subdept_delete_guard”

	демонстрация text	статус text	сообщение text
1	удаление подразделения 1	отклонено (ожидаемо)	lr7: подразделение 1 (ООО «Промтех») нельзя удалить: есть дочернее подразделение с ид 2

Рисунок 10 – Результаты работы программы

Далее файл 06_demo_triggers.sql, блок DO демонстрирует срабатывание триггера на “прием”: вставка с датой приёма в будущем перехватывается в EXCEPTION и выводится NOTICE. Отдельно в транзакции BEGIN ... UPDATE ... SELECT из lr7_employee_audit ... ROLLBACK показано появление строки аудита до отката транзакции. Результаты представлены на рисунках 11 – 12.

```

SELECT
  obj.имя AS объект,
  CASE
    WHEN obj.тип = 'table' AND t.table_name IS NOT NULL THEN 'есть'
    WHEN obj.тип = 'trigger' AND tr.trigger_name IS NOT NULL THEN 'есть'
    ELSE 'НЕТ – выполните скрипты 01-05'
  END AS статус
FROM (
  VALUES
    ('lr7_employee_audit', 'table'),
    ('lr7_trg_priyom_date_check', 'trigger'),
    ('lr7_trg_employee_audit', 'trigger'),
    ('lr7_trg_deti_limit', 'trigger'),
    ('lr7_trg_subdept_delete_guard', 'trigger')
) AS obj(имя, тип)
LEFT JOIN information_schema.tables t
  ON obj.тип = 'table' AND t.table_schema = 'public' AND t.table_name = obj.имя
LEFT JOIN information_schema.triggers tr
  ON obj.тип = 'trigger' AND tr.trigger_schema = 'public' AND tr.trigger_name = obj.имя;

DROP TABLE IF EXISTS _lr7_demo_result;
CREATE TEMP TABLE _lr7_demo_result (
  n int NOT NULL,
  демонстрация text NOT NULL,
  статус text NOT NULL,
  сообщение text
);

```

Рисунок 11 – Процедура “demo_triggers”

	trigger_name name	action_timing character varying	event_manipulation character varying
1	lr7_trg_deti_limit	BEFORE	INSERT
2	lr7_trg_employee_audit	AFTER	DELETE
3	lr7_trg_employee_audit	AFTER	INSERT
4	lr7_trg_employee_audit	AFTER	UPDATE
5	lr7_trg_priyom_date_check	BEFORE	INSERT
6	lr7_trg_priyom_date_check	BEFORE	UPDATE
7	lr7_trg_subdept_delete_gu...	BEFORE	DELETE

Рисунок 12 – Результаты работы программы

4 Вывод

Триггеры позволяют централизовать проверки и побочные эффекты на уровне СУБД. BEFORE-триггеры удобны для запрета некорректных данных, AFTER – для несвязанных с откатом вставок в журнал. Недостатки: усложнение отладки и необходимость помнить о порядке срабатывания при нескольких триггерах.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

```
-- ===== --
-- 01_aux_audit_table.sql
-- ===== --

CREATE TABLE IF NOT EXISTS lr7_employee_audit (
    id bigserial PRIMARY KEY,
    ts timestamptz NOT NULL DEFAULT clock_timestamp(),
    op text NOT NULL,
    emp_id int,
    fio_old varchar(200),
    fio_new varchar(200),
    birth_old date,
    birth_new date,
    CONSTRAINT lr7_employee_audit_op_chk CHECK (op IN ('INSERT',
'UPDATE', 'DELETE'))
);

COMMENT ON TABLE lr7_employee_audit IS 'ЛР7: журнал DML по таблице
сотрудник (триггер AFTER).';

-- ===== --
-- 02_trg_priyom_date_check.sql
-- ===== --

CREATE OR REPLACE FUNCTION lr7_fn_priyom_date_check()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.дата_приема > CURRENT_DATE THEN
```

```

        RAISE EXCEPTION 'lr7: дата приёма не может быть позже текущей даты
(получено: %)', NEW.дата_приема
        USING ERRCODE = '23514'; -- check_violation
    END IF;
    RETURN NEW;
END;
$$;

```

```

DROP TRIGGER IF EXISTS lr7_trg_priyom_date_check ON прием;

```

```

CREATE TRIGGER lr7_trg_priyom_date_check
    BEFORE INSERT OR UPDATE OF дата_приема ON прием
    FOR EACH ROW
    EXECUTE PROCEDURE lr7_fn_priyom_date_check();

```

```

-- =====
-- 03_trg_employee_audit.sql
-- =====

```

```

CREATE OR REPLACE FUNCTION lr7_fn_employee_audit()
    RETURNS trigger
    LANGUAGE plpgsql
    AS $$
    BEGIN
        IF TG_OP = 'DELETE' THEN
            INSERT INTO lr7_employee_audit (op, emp_id, fio_old, birth_old)
            VALUES ('DELETE', OLD.ид, OLD.фιο, OLD.дата_рождения);
            RETURN OLD;
        ELSIF TG_OP = 'UPDATE' THEN
            INSERT INTO lr7_employee_audit (op, emp_id, fio_old, fio_new, birth_old,
            birth_new)

```

```

VALUES ('UPDATE', NEW.ид, OLD.фio, NEW.фio, OLD.дата_рождения,
NEW.дата_рождения);
RETURN NEW;
ELSIF TG_OP = 'INSERT' THEN
INSERT INTO lr7_employee_audit (op, emp_id, fio_new, birth_new)
VALUES ('INSERT', NEW.ид, NEW.фio, NEW.дата_рождения);
RETURN NEW;
END IF;
RETURN NULL;
END;
$$;

```

```

DROP TRIGGER IF EXISTS lr7_trg_employee_audit ON сотрудник;
CREATE TRIGGER lr7_trg_employee_audit
AFTER INSERT OR UPDATE OR DELETE ON сотрудник
FOR EACH ROW
EXECUTE PROCEDURE lr7_fn_employee_audit();

```

```

-- =====
-- 04_trg_deti_limit.sql
-- =====

CREATE OR REPLACE FUNCTION lr7_fn_deti_limit()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
v_cnt int;
v_max constant int := 12;
BEGIN

```

```

SELECT COUNT(*)::int INTO v_cnt
FROM дети
WHERE ид_сотрудника = NEW.ид_сотрудника;

IF v_cnt >= v_max THEN
    RAISE EXCEPTION 'lr7: у сотрудника % уже % детей в таблице (лимит
%),
    NEW.ид_сотрудника, v_cnt, v_max
    USING ERRCODE = '23514';
END IF;

RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS lr7_trg_deti_limit ON дети;

CREATE TRIGGER lr7_trg_deti_limit
    BEFORE INSERT ON дети
    FOR EACH ROW
    EXECUTE PROCEDURE lr7_fn_deti_limit();

-- ===== --
-- 05_trg_subdept_delete_guard.sql
-- ===== --

CREATE OR REPLACE FUNCTION lr7_fn_subdept_delete_guard()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE

```

```

v_child int;
BEGIN
  SELECT п.ид INTO v_child
  FROM подразделение п
  WHERE п.ид_родительское = OLD.ид
  LIMIT 1;

  IF v_child IS NOT NULL THEN
    RAISE EXCEPTION 'lr7: подразделение % (%) нельзя удалить: есть
дочернее подразделение с ид %',
    OLD.ид, OLD.наименование, v_child
    USING ERRCODE = '23503'; -- foreign_key_violation
  END IF;

  RETURN OLD;
END;
$$;

DROP TRIGGER IF EXISTS lr7_trg_subdept_delete_guard ON подразделение;

CREATE TRIGGER lr7_trg_subdept_delete_guard
  BEFORE DELETE ON подразделение
  FOR EACH ROW
  EXECUTE PROCEDURE lr7_fn_subdept_delete_guard();

-- ===== --
-- 06_demo_triggers.sql
-- ===== --
-- ЛР7: сводная демонстрация триггеров. Выполнять ПОСЛЕ 01, 02, 03, 04, 05
(по порядку).

```

```

-- NOTICE «_lr7_demo_result does not exist» при первом DROP — норма.

-- ==== Проверка подготовки (Data Output) ====
SELECT
    obj.имя AS объект,
    CASE
        WHEN obj.тип = 'table' AND t.table_name IS NOT NULL THEN 'есть'
        WHEN obj.тип = 'trigger' AND tr.trigger_name IS NOT NULL THEN 'есть'
        ELSE 'НЕТ — выполните скрипты 01–05'
    END AS статус
FROM (
    VALUES
        ('lr7_employee_audit', 'table'),
        ('lr7_trg_priyom_date_check', 'trigger'),
        ('lr7_trg_employee_audit', 'trigger'),
        ('lr7_trg_deti_limit', 'trigger'),
        ('lr7_trg_subdept_delete_guard', 'trigger')
) AS obj(имя, тип)
LEFT JOIN information_schema.tables t
    ON obj.тип = 'table' AND t.table_schema = 'public' AND t.table_name = obj.имя
LEFT JOIN information_schema.triggers tr
    ON obj.тип = 'trigger' AND tr.trigger_schema = 'public' AND tr.trigger_name =
obj.имя;

DROP TABLE IF EXISTS _lr7_demo_result;
CREATE TEMP TABLE _lr7_demo_result (
    n int NOT NULL,
    демонстрация text NOT NULL,
    статус text NOT NULL,
    сообщение text

```

);

-- 1) BEFORE прием: дата в будущем

DO \$\$

BEGIN

INSERT INTO прием (ид, ид_сотрудника, ид_подразделения, ид_должности,
дата_приема, совместительство)

VALUES (900, 1, 2, 1, CURRENT_DATE + 1, B'0');

INSERT INTO _lr7_demo_result

VALUES (1, 'дата приёма в будущем', 'неожиданно: вставка прошла', NULL);

EXCEPTION

WHEN others THEN

INSERT INTO _lr7_demo_result

VALUES (1, 'дата приёма в будущем', 'отклонено (ожидаемо)',

SQLERRM);

END;

\$\$;

-- 2) AFTER сотрудник → lr7_employee_audit

DO \$\$

DECLARE

v_cnt bigint;

BEGIN

IF NOT EXISTS (

SELECT 1 FROM information_schema.tables

WHERE table_schema = 'public' AND table_name = 'lr7_employee_audit'

) THEN

INSERT INTO _lr7_demo_result

VALUES (2, 'аудит сотрудника (UPDATE)', 'пропущено', 'нет таблицы —
выполните 01_aux_audit_table.sql и 03_trg_employee_audit.sql');


```

RETURN;
END IF;

UPDATE сотрудник SET фιο = TRIM(фιο) WHERE ид = 8;
UPDATE сотрудник SET фιο = фιο || ' ' WHERE ид = 8;

SELECT COUNT(*) INTO v_cnt FROM lr7_employee_audit;

INSERT INTO _lr7_demo_result
VALUES (
    2,
    'аудит сотрудника (UPDATE)',
    'в журнале есть записи',
    'строк в lr7_employee_audit: ' || v_cnt::text
);
EXCEPTION
    WHEN others THEN
        INSERT INTO _lr7_demo_result
        VALUES (2, 'аудит сотрудника (UPDATE)', 'ошибка', SQLERRM);
END;
$$;

-- 3) BEFORE дети: лимит 12
DO $$
BEGIN
    INSERT INTO дети (ид, ид_сотрудника, фιο, пол, дата_рождения)
    VALUES (900, 6, 'LR7 лишний ребёнок', 'M', '2011-01-01');
    INSERT INTO _lr7_demo_result
    VALUES (3, '13-й ребёнок у сотрудника 6', 'неожиданно: вставка прошла',
    NULL);

```

```

EXCEPTION
    WHEN others THEN
        INSERT INTO _lr7_demo_result
            VALUES (3, '13-й ребёнок у сотрудника 6', 'отклонено (ожидаемо)',
SQLERRM);
END;
$$;

```

-- 4) BEFORE подразделение: удаление «Промтех»

```

DO $$
BEGIN
    DELETE FROM подразделение WHERE ид = 1;
    INSERT INTO _lr7_demo_result
        VALUES (4, 'удаление подразделения 1', 'неожиданно: удалено', NULL);
EXCEPTION
    WHEN others THEN
        INSERT INTO _lr7_demo_result
            VALUES (4, 'удаление подразделения 1', 'отклонено (ожидаемо)',
SQLERRM);
END;
$$;

```

-- === Результат выполнения скрипта (Data Output) ===

```

SELECT * FROM _lr7_demo_result ORDER BY n;

```

```

DROP TABLE IF EXISTS _lr7_audit_show;

```

```

DO $q$

```

```

BEGIN

```

```

    IF EXISTS (

```

```

        SELECT 1 FROM information_schema.tables

```

```

WHERE table_schema = 'public' AND table_name = 'lr7_employee_audit'
) THEN
EXECUTE $$
CREATE TEMP TABLE _lr7_audit_show AS
SELECT id, ts, op, emp_id, fio_old, fio_new
FROM lr7_employee_audit
ORDER BY id DESC
LIMIT 5
$$;
ELSE
CREATE TEMP TABLE _lr7_audit_show (
id bigint, ts timestamptz, op text, emp_id int,
fio_old varchar(200), fio_new varchar(200)
);
INSERT INTO _lr7_audit_show (op, fio_new)
VALUES ('—', 'выполните 01 и 03, затем снова 06');
END IF;
END;
$q$;

SELECT * FROM _lr7_audit_show;

SELECT trigger_name, action_timing, event_manipulation
FROM information_schema.triggers
WHERE trigger_schema = 'public' AND trigger_name LIKE 'lr7_%'
ORDER BY trigger_name, event_manipulation;

```